

1. (A) (i) The following is the formula generated from the linear regressor model from Listing 1.

$$y = + 86.34x_{10} + 639.10x_9 + 147.19x_8 - 33.62x_7 + 213.10x_6 - 475.57x_5 \\ + 293.68x_4 + 513.86x_3 - 248.85x_2 - 8.07x_1 + 152.06$$

- (ii) Below is the corresponding implementation:

```
1 import numpy as np
2 import sklearn.datasets
3
4 from sklearn.model_selection import train_test_split
5 from sklearn.linear_model import LinearRegression
6
7 X, y = sklearn.datasets.load_diabetes(return_X_y=True, as_frame=False)
8 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
9
10 model = LinearRegression().fit(X_train, y_train)
11 coefs = np.insert(model.coef_, 0, model.intercept_)
12 print(coefs)
```

Listing 1. linreg.py

- (B) (i) The mean squared error across the training set is approximately 2850.26.
- (ii) The mean squared error across the testing set is approximately 2929.90.
- (iii) Because the test error is computed on an unknown dataset that the model has not seen.
- (iv) Below is the corresponding implementation:

```
1 import numpy as np
2 import sklearn.datasets
3
4 from sklearn.model_selection import train_test_split
5 from sklearn.linear_model import LinearRegression
6 from sklearn.metrics import mean_squared_error
7
8 X, y = sklearn.datasets.load_diabetes(return_X_y=True, as_frame=False)
9 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
10
11 model = LinearRegression().fit(X_train, y_train)
12 train_error = mean_squared_error(y_train, model.predict(X_train))
13 test_error = mean_squared_error(y_test, model.predict(X_test))
14 print(train_error, test_error)
```

Listing 2. linreg-mse.py

- (C) (i) The following is the formula generated from the polynomial regressor model from Listing 3.

$$\begin{aligned}
 y = & +709.24x_{66} - 1408.68x_{65} + 52113.48x_{64} + 3352.74x_{63} + 2680.27x_{62} + 14582.90x_{61} \\
 & - 3184.94x_{60} - 161235.03x_{59} + 20538.04x_{58} - 14138.74x_{57} - 13063.50x_{56} - 374684.71x_{55} \\
 & + 3038.02x_{54} - 85714.57x_{53} - 86908.47x_{52} + 14320.74x_{51} + 415280.33x_{50} - 25627.73x_{49} \\
 & + 74622.62x_{48} + 178770.98x_{47} - 81807.58x_{46} - 3688.90x_{45} + 7004.13x_{44} - 278.10x_{43} \\
 & + 6098.79x_{42} + 15406.29x_{41} - 15940.76x_{40} - 127.95x_{39} - 89.55x_{38} + 6865.90x_{37} \\
 & - 4972.56x_{36} + 3886.12x_{35} + 15382.69x_{34} - 16388.40x_{33} + 1842.70x_{32} + 2192.86x_{31} \\
 & + 663.52x_{30} - 3962.50x_{29} - 994.72x_{28} - 2201.34x_{27} - 10411.20x_{26} + 10610.84x_{25} \\
 & + 2106.24x_{24} + 1982.70x_{23} - 1.50x_{22} + 547.49x_{21} + 7403.02x_{20} + 4663.99x_{19} \\
 & + 10766.94x_{18} + 10944.90x_{17} - 17855.84x_{16} + 629.37x_{15} - 582.15x_{14} + 3402.24x_{13} \\
 & + 2114.09x_{12} + 88.24x_{11} + 34633.80x_{10} + 40.56x_9 + 38357.87x_8 + 90595.51x_7 \\
 & - 103220.46x_6 + 294.70x_5 + 365.65x_4 - 247.91x_3 + 41.48x_2 + 0.00x_1 - 313.90
 \end{aligned}$$

- (ii) Below is the corresponding implementation.

```

1  import numpy as np
2  import sklearn.datasets
3
4  from sklearn.model_selection import train_test_split
5  from sklearn.linear_model import LinearRegression
6  from sklearn.preprocessing import PolynomialFeatures
7
8  X, y = sklearn.datasets.load_diabetes(return_X_y=True, as_frame=False)
9  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
10
11 X_train = PolynomialFeatures(degree=2, include_bias=False).fit_transform(X_train)
12
13 model = LinearRegression().fit(X_train, y_train)
14 coefs = np.insert(model.coef_, 0, model.intercept_)
15 print(coefs)

```

Listing 3. polyreg.py

- (D) (i) The mean squared error across the training set is approximately 2325.51.
- (ii) The mean squared error across the testing set is approximately 3772.52.
- (iii) Compared to the canonical Linear Regressor from 1(B), the Polynomial Regressor is catastrophically overfitting the training observations and generalizing poorly on the test set.
- (E) (i) Below is an implementation of Ridge Regression for a singular value of α :

```

1  import numpy as np
2  import sklearn.datasets
3
4  from sklearn.model_selection import train_test_split
5  from sklearn.linear_model import Ridge
6  from sklearn.metrics import mean_squared_error
7  from sklearn.preprocessing import PolynomialFeatures
8
9  X, y = sklearn.datasets.load_diabetes(return_X_y=True, as_frame=False)
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
11

```

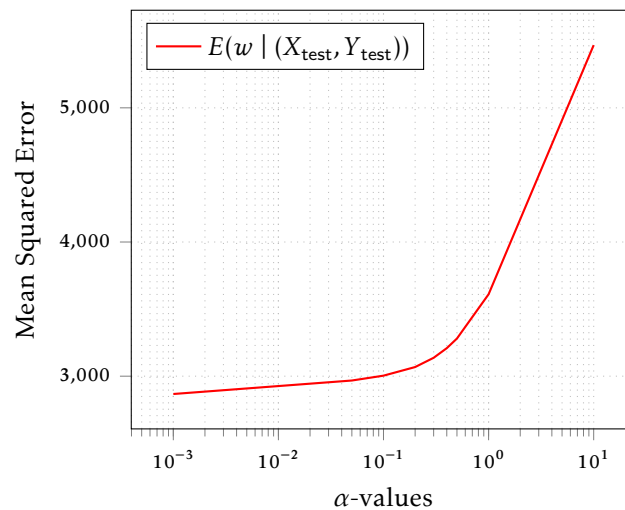
```

12 polyf = PolynomialFeatures(degree=2, include_bias=True)
13 X_train = polyf.fit_transform(X_train)
14 X_test = polyf.fit_transform(X_test)
15
16 model = Ridge(alpha=0.0, fit_intercept=False).fit(X_train, y_train)
17 error = mean_squared_error(y_test, model.predict(X_test))
18 print(error)

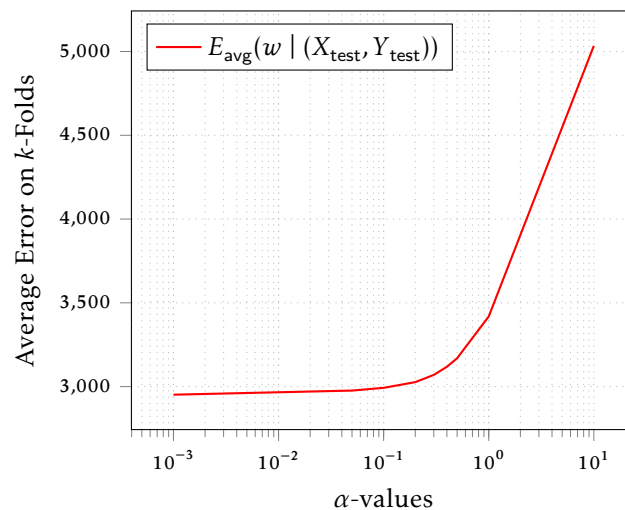
```

Listing 4. ridge.py

- (ii) Below is a plot with the regularization factor, α , on the x -axis (log scale) and the mean squared error (without regularization penalty) on the y -axis. See Listing 6 for a fuller explanation on how this figure has been generated.



- (F) (i) Below is a plot with the regularization factor, α , on the x -axis (log scale) and the mean test error across the five splits on the y -axis. See Listing 7 for a fuller explanation on how this figure has been generated.



- (ii) After tuning α via cross-validation, we should evaluate the model on a separate test set that was

not involved in training or hyperparameter tuning. This yields a less biased estimate of how well the model performs on unseen data.

2. The matching pursuit algorithm that recovers the non-zero entries of the sparse signal vector S is shown below along with the required ordered list of the entries and the residual vector norm after each step.

```

1  import numpy as np
2
3  from numpy.linalg import norm
4
5  M_DATA = 'data/M.csv'
6  W_DATA = 'data/W.csv'
7
8  def load_dataset(dataset: str) → np.ndarray:
9      data = np.loadtxt(dataset, delimiter=',')
10     return data
11
12  def matching_pursuit(X_mat: np.ndarray, y_vec: np.ndarray, s_val: float = 0.0, k_val: int = 69) → np.
    ndarray:
13      r_vec = y_vec
14      alpha = np.zeros(X_mat.shape[1])
15
16      for _ in range(k_val):
17          r_cov = r_vec @ X_mat
18          j_idx = np.argmax(np.abs(r_cov))
19
20          alph1 = (r_vec @ X_mat[:, j_idx] + 0.5 * s_val) / np.square(norm(X_mat[:, j_idx]))
21          alph2 = (r_vec @ X_mat[:, j_idx] - 0.5 * s_val) / np.square(norm(X_mat[:, j_idx]))
22          norm1 = norm(r_vec - X_mat[:, j_idx] * alph1) + s_val * np.abs(alph1)
23          norm2 = norm(r_vec - X_mat[:, j_idx] * alph2) + s_val * np.abs(alph2)
24
25          alphj = alph1 if norm1 < norm2 else alph2
26          r_vec = r_vec - X_mat[:, j_idx] * alphj
27          alpha[j_idx] = alphj
28
29          print(alphj, norm(r_vec))
30
31      return alpha
32
33  def main() → int:
34      M = load_dataset(M_DATA)
35      W = load_dataset(W_DATA)
36      S = matching_pursuit(M, W)
37      return 0
38
39  if __name__ == '__main__':
40      raise SystemExit(main())

```

Listing 5. matching-pursuit.py

Appendix

```
1 import numpy as np
2 import sklearn.datasets
3
4 from sklearn.model_selection import train_test_split
5 from sklearn.linear_model import Ridge
6 from sklearn.metrics import mean_squared_error
7 from sklearn.preprocessing import PolynomialFeatures
8
9 X, y = sklearn.datasets.load_diabetes(return_X_y=True, as_frame=False)
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
11
12 polyf = PolynomialFeatures(degree=2, include_bias=True)
13 X_train = polyf.fit_transform(X_train)
14 X_test = polyf.fit_transform(X_test)
15
16 alpha_vals = np.array([0.0, 0.001, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 1.0, 10.0])
17 error_vals = np.zeros_like(alpha_vals)
18
19 for i in range(len(alpha_vals)):
20     alpha = alpha_vals[i]
21     model = Ridge(alpha=alpha, fit_intercept=False).fit(X_train, y_train)
22     error = mean_squared_error(y_test, model.predict(X_test))
23     error_vals[i] = error
24
25 np.savetxt('data/alpha-vs-error.csv', np.column_stack((alpha_vals, error_vals)), delimiter=',')
```

Listing 6. ridge-mse.py

```
1 import numpy as np
2 import sklearn.datasets
3
4 from sklearn.model_selection import KFold
5 from sklearn.linear_model import Ridge
6 from sklearn.metrics import mean_squared_error
7 from sklearn.preprocessing import PolynomialFeatures
8
9 num_splits = 5
10 alpha_vals = np.array([0.0, 0.001, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 1.0, 10.0])
11 error_vals = np.zeros_like(alpha_vals)
12 error_fold = np.zeros(num_splits)
13
14 X, y = sklearn.datasets.load_diabetes(return_X_y=True, as_frame=False)
15 X = PolynomialFeatures(degree=2, include_bias=True).fit_transform(X)
16
17 fold = KFold(n_splits=num_splits)
18 fold.get_n_splits(X)
19
20 for i in range(len(alpha_vals)):
21     alpha = alpha_vals[i]
22     model = Ridge(alpha=alpha, fit_intercept=False)
23
24     for j, (train_idx, test_idx) in enumerate(fold.split(X)):
25         X_train, X_test, y_train, y_test = X[train_idx], X[test_idx], y[train_idx], y[test_idx]
26         model = model.fit(X_train, y_train)
27         error = mean_squared_error(y_test, model.predict(X_test))
28         error_fold[j] = error
29
30     error_vals[i] = error_fold.mean()
31
32 np.savetxt('data/alpha-vs-error-kfold.csv', np.column_stack((alpha_vals, error_vals)), delimiter=',')
```

Listing 7. ridge-kfold.py